

Guitar FX Pedalboard

System Architecture & Signal-Processing Reference

This document describes, in detail, how the browser-based Guitar FX Pedalboard application works internally: how it generates pedal and amplifier effects, how it reduces noise, how it records and exports audio, and how the supporting subsystems (tuner, drum machine, UI) are built. The entire application runs client-side in the browser using the Web Audio API — there is no backend server, no native audio engine, and no third-party audio/DSP library. Every effect, every amplifier voicing, every reverb/cabinet impulse response, and every drum sound is generated procedurally in JavaScript.

Source root: `guitar-fx-pedalboard/`

Key files: `js/audio-engine.js`, `js/pedal-registry.js`, `js/amp-registry.js`, `js/ir-synth.js`, `js/noise-gate-worklet.js`, `js/spectral-denoise-worklet.js`, `js/dynamics-worklet.js`, `js/bitcrusher-worklet.js`, `js/pitch-worklet.js`, `js/pluck-worklet.js`, `js/looper.js`, `js/wav-encoder.js`, `js/tuner.js`, `js/drum-kit.js`, `js/ui.js`, `js/main.js`

1. High-Level Architecture

The application is a single-page static web app (index.html + ES modules under js/). There is no build step, bundler, or server-side component — every module is loaded directly by the browser via native `<script type="module">` imports. All real-time audio processing happens inside one shared `AudioContext`, constructed and managed by the `AudioEngine` class in `js/audio-engine.js`.

1.1 Signal flow

Audio flows through a fixed backbone graph, with a variable-length user-configurable pedal chain and a single optional amplifier spliced into the middle of it:

```
Guitar input (getUserMedia) -> inputGain (input trim / mute) +-> scopeAnalyser (oscilloscope UI) +-> tunerAnalyser (chromatic tuner) \-> spectralDenoiseNode (AudioWorklet: spectral noise reduction) -> noiseGateNode (AudioWorklet: envelope-follower gate) -> preChainTap (stable side-chain tap point) -> [ user's pedal chain, in order, each may be bypassed ] -> [ optional single amplifier ] -> [ optional acoustic-body simulator ] -> masterGain (master volume) -> outputLimiter (WaveShaper safety limiter) +-> ctx.destination (speakers/headphones) \-> mediaStreamDest (tapped for recording)
```

1.2 Why AudioWorklets instead of native nodes

Several effects (noise gate, spectral denoiser, compressor/limiter, bit crusher, octaver, pitch shifter, the Karplus-Strong tuner tone) are implemented as custom **AudioWorkletProcessors** — hand-written JavaScript classes that run sample-by-sample on the browser's dedicated real-time audio rendering thread, registered via `ctx.audioWorklet.addModule()`. This is necessary because native Web Audio nodes either don't support the required behavior (sample-and-hold, custom envelope followers, delay-line pitch shifting, FFT-based spectral processing) or, in the case of `DynamicsCompressorNode`, carry a fixed ~6 ms look-ahead buffer baked into every browser's implementation that cannot be disabled or configured. A recurring design decision throughout this codebase is to reimplement compression as a hand-written, zero-look-ahead worklet (see §4) purely to remove that involuntary latency from the signal path, at the honest cost of occasional transient overshoot instead of a hidden delay.

1.3 Input handling

- Captured via `navigator.mediaDevices.getUserMedia` with `echoCancellation`, `noiseSuppression` and `autoGainControl` all explicitly disabled — the browser's own speech-oriented processing would otherwise mangle a guitar signal.
- Requests latency: `{ideal: 0}` and a capture sample rate matching the `AudioContext`'s own rate, to minimize buffering and avoid an extra internal resample step.
- The `AudioContext` itself is created with `latencyHint: 0`, asking the browser for the smallest safe buffer size instead of the conservative default.
- Stereo input (`channelCount` ideal: 2) is requested and explicitly downmixed via a `ChannelSplitterNode` feeding both channels into the mono input-gain stage — requesting mono directly caused some audio interfaces to silently drop channel 2.
- Output device selection uses the experimental `AudioContext.setSinkId()` API (Chrome 110+), and can auto-match the output device to the input device by comparing `MediaDeviceInfo.groupId` (same physical USB interface).

1.4 The always-on output limiter

Because pedal/amp combinations vary hugely in loudness (a cranked Fuzz or high-gain amp can be many times louder than a filter-heavy Wah), the engine always terminates the chain in a safety limiter before `ctx.destination`. This is implemented as a `WaveShaperNode` with a hand-built soft-knee curve (linear below -6 dBFS, 20:1 compressed above it, hard-clamped at full scale) rather than the native `DynamicsCompressorNode`, for the same look-ahead-latency reason described above — a waveshaper is a memoryless per-sample transfer function, so it adds exactly zero latency to every user's signal, at the cost of instant saturation on rare peaks instead of a smooth, time-based gain reduction.

2. Pedal Effects Engine (js/pedal-registry.js)

Every stompbox is a plain JavaScript object in a single array (PEDAL_TYPES), each shaped as: { id, label, category, color, blurb, about, createNodes(ctx), params[] }. createNodes(ctx) wires together Web Audio nodes and returns { input, output, nodes } — the entry/exit points the engine splices into the chain, and a handle to the internal nodes each parameter's apply(nodes, value, ctx) function will later mutate live as the user turns a knob. There are 20 pedals across 7 categories: Dynamics, Drive, Filter/Wah, Modulation, Pitch, Time-Based and EQ.

2.1 Shared building blocks

- **driveCurve(k, bias)** — generates a 4096-point WaveShaperNode transfer curve using $\tanh(k \times (x + \text{bias}))$, offset so zero input still maps to zero output. This single function is the clipping/saturation core shared by every drive pedal *and* every amplifier's preamp stage — the only things that differ between Overdrive, Distortion, Fuzz, and each amp voicing are the value of k (drive amount), an optional DC bias (for asymmetric, "tube-sag"-style clipping), and the WaveShaper's oversample setting (none/2x/4x, trading CPU for less aliasing on harder-driven curves).
- **createModDelay()** — a shared topology (DelayNode + sine LFO modulating delay time + feedback + dry/wet mix) used for both Chorus and Flanger; they differ only in delay time, modulation depth/rate, and feedback amount.
- **setMix / setLevel** — small shared helpers wiring a 0–100% knob to a wet/dry gain pair or a single output gain, reused across most pedals instead of being re-implemented per pedal.

2.2 Dynamics

- **Compressor** — the custom zero-look-ahead dynamics-processor AudioWorklet (see §4) followed by a makeup-gain node. Threshold, ratio, attack, release and makeup level are all live-mapped worklet parameters.
- **Clean Boost** — a plain GainNode (level) feeding a BiquadFilterNode high-shelf (tone) — a transparent gain stage with no distortion.
- **Limiter** — the same dynamics-processor worklet, pre-configured with a fast 1 ms attack, 20:1 ratio and a tight 2 dB knee, exposing only Ceiling (threshold), Release and Level to the user.

2.3 Drive

- **Overdrive** — driveCurve(1 + drive×0.14) through a 2x-oversampled WaveShaper, a lowpass tone filter, and a level gain. Moderate k keeps it touch-sensitive.
- **Distortion** — a harder curve (4 + drive×0.4, 4x oversample) plus a peaking mid-boost filter ahead of the tone lowpass, for a thicker, more compressed clip.
- **Fuzz** — the hardest/most asymmetric curve of the three (8 + fuzz×0.45 with a 0.18 DC bias), 4x oversampled, fixed-frequency lowpass tone.
- **Bit Crusher** — a custom bitcrusher-processor AudioWorklet (see §4) doing true sample-and-hold (rate reduction) and amplitude quantization (bit-depth reduction) — digital/lo-fi degradation rather than analog-style clipping.

2.4 Filter / Wah

- **Wah** — a resonant bandpass BiquadFilterNode whose center frequency is directly controlled by the "Treadle" knob, standing in for a real wah pedal's rocking motion.
- **Auto-Wah** — the same bandpass filter, but its frequency is swept automatically by a sine LFO instead of the user's foot.

- **Envelope Filter** — the bandpass filter's frequency is driven by a custom envelope-follower-processor AudioWorklet (defined alongside the noise gate in noise-gate-worklet.js) that tracks the input's amplitude envelope and outputs it as an audio-rate control signal — this is the only pedal that reads from a dedicated side-chain input (`sideChainInput`), tapped straight off the pre-chain signal so its response tracks playing dynamics, not the wet/processed signal.

2.5 Modulation

- **Chorus / Flanger** — both built from the shared `createModDelay()` topology, differing only in constants (Chorus: longer delay ~20ms, slower/shallower modulation, low feedback; Flanger: very short delay ~3ms, slower LFO but deeper modulation, much higher feedback for its metallic comb-filtered sweep).
- **Phaser** — six cascaded all-pass BiquadFilterNode stages, each frequency-modulated in parallel by one shared sine LFO, with feedback from the last stage back to the first and a dry/wet mix.
- **Tremolo** — a single GainNode whose gain is modulated by a sine LFO (volume pulsing, not pitch).
- **Vibrato** — a very short DelayNode whose delay time is modulated by a sine LFO, producing pitch wobble instead of volume wobble (easy to confuse with Tremolo, but a different modulation target entirely).
- **Ring Modulator** — a sine-wave carrier oscillator multiplies the input via a GainNode (the oscillator drives the target gain's own `.gain` AudioParam), producing inharmonic sum/difference frequencies rather than following the input's own harmonic series.
- **Rotary / Leslie** — three synchronized sine/triangle LFOs simultaneously modulate amplitude (via gain), a short delay (subtle pitch/Doppler effect), and a StereoPannerNode's pan position, emulating a rotating speaker cabinet's combined amplitude/pitch/stereo motion.

2.6 Pitch

- **Octaver** — a custom octaver-processor AudioWorklet doing analog-style octave-down synthesis via zero-crossing frequency division: it tracks upward zero-crossings and flips a square wave's sign every other crossing (halving the fundamental frequency), smooths that square wave, scales it by an envelope follower tracking the input's amplitude, and mixes it back with the dry signal.
- **Pitch Shifter** — a custom pitch-shifter-processor AudioWorklet: a classic dual-tap delay-line/granular shifter. A 2048-sample circular buffer is read by two independent read pointers advancing at the target pitch ratio ($2^{(\text{semitones}/12)}$) instead of 1 sample/sample; the two taps are triangle-windowed and crossfaded against each other so that whichever tap is mid-buffer (away from its wrap-around discontinuity) dominates the output, avoiding audible clicks. This has inherent latency of roughly half the buffer (~21–23 ms), an unavoidable cost of this technique.

2.7 Time-Based

- **Delay** — a standard DelayNode with a lowpass tone filter inside the feedback loop (so repeats darken progressively, like a real analog/tape delay) and a dry/wet mix.
- **Reverb** — a ConvolverNode loaded with a procedurally synthesized impulse response (never a recorded sample) — see §5 for how each reverb "type" (Room/Plate/Hall/Spring) is built.

2.8 EQ

- **Exciter** — a parallel harmonic-exciter ("Aphex Aural Exciter"-style) circuit: the dry signal passes straight through, while a parallel branch high-pass filters just the top end, drives it through a dedicated WaveShaper curve to generate new harmonic content, low-pass filters that back down to keep it musical (not harsh/aliased), and blends it back in — adding clarity via new overtones rather than just boosting existing treble (and its noise) with a plain EQ band.

- **Graphic EQ** — six cascaded peaking `BiquadFilterNodes` at fixed standard frequencies (100/250/630/1600/4000/10000 Hz), each independently gain-controlled.

3. Amplifier Engine (js/amp-registry.js)

Every amplifier voicing is built by one shared factory, `createAmpNodes(ctx, cfg)`, so an "amp" is really a fixed preset pedal chain under the hood, using the exact same `driveCurve()` WaveShaper technique as the drive pedals:

```
inputGain (Gain knob) -> [optional built-in compressor worklet] -> preEQ band(s) (highpass  
rumble filter, optional pre-drive midrange shaping) -> WaveShaper (fixed per-amp  
driveCurve(curveK, curveBias)) -> postEQ bands (tone shaping after clipping; one band doubles  
as Presence) -> cabConvolver (synthesized cabinet impulse response) -> outputLevel (Level  
knob)
```

The **Gain** knob doesn't just add loudness — it scales `inputGain` within a per-amp `driveRange` (e.g. Clean: 0.4–1.6×, Lead: 6–26×), i.e. it controls exactly how hard the signal is driven into the *fixed* clipping curve, mimicking how a real tube preamp's gain stage works. **Presence** scales one specific post-EQ band's gain (a treble/high-shelf-region peaking filter). **Level** is a plain post-cabinet output trim.

3.1 Standard amps

- **Clean** — very low drive range (0.4–1.6×) that never reaches the clipping threshold even at maximum, so no distortion harmonics are added at any Gain setting; a flat-ish, slightly presence-boosted post-EQ and the cabinet response are the only tone shaping.
- **Crunch** — a moderate drive range (2–10×) that sits right at the edge of breakup: soft picking stays fairly clean, digging in pushes past the clipping knee for classic rock rhythm grit.
- **Lead / High-Gain** — a hot drive range (6–26×) into a steep curve ($k=10$, 4x oversampled) with scooped mids and boosted presence in the post-EQ, built for sustain on solos and heavier riffing.

3.2 Signature amps

Four additional band-inspired voicings layer on top of the same engine, differing only in their config numbers (drive range, curve steepness/bias, EQ curve, and cabinet resonance):

- **AC/DC** — tight, fairly symmetric clipping ($k=7$, `driveRange` 3–14×) with a highpassed low end and boosted upper-mid peak, so individual notes in a chord stay distinct instead of smearing under gain — a Marshall-plexi-style aggressive rhythm crunch. Uses 4x oversampling (raised from an earlier 2x) because its drive range peaks nearly as high as Lead's and needed more anti-aliasing headroom to avoid audible "static" on the clipped harmonics.
- **Led Zeppelin** — the only amp using an asymmetric clipping curve (`curveBias` = 0.035), which generates even-order harmonics for a softer, "sagging" vintage breakup instead of a tight symmetric crunch, with a broad woody midrange bump and a lower drive range (2–10×) than AC/DC.
- **Oasis** — the hottest of the four (`driveRange` 8–30×, $k=12$), a scooped-mid / boosted-treble "wall of sound" EQ curve designed to stay readable even with multiple layered rhythm-guitar tracks stacked on top.
- **Dire Straits** — barely reaches the clipping threshold at all (`driveRange` 0.6–3×, the lowest of any amp) — almost no distortion. Its character instead comes from a built-in compressor (the same dynamics-processor worklet, fixed 4:1 ratio) evening out picking dynamics, plus a scooped-mid, boosted-treble EQ curve for the glassy single-coil "quack."

3.3 Acoustic-body simulator (post-chain, independent toggle)

A separate always-optional block, `createAcousticSimNodes()`, applied after the entire pedal+amp chain (not part of it): two peaking-EQ cuts (removing some of an electric signal's low-mid boxiness) feeding a `ConvolverNode` loaded with a synthesized "acoustic body" impulse response, blended 15% dry / 85% wet.

4. Custom Signal-Processing Worklets (the low-level DSP)

These are the from-scratch AudioWorkletProcessor classes doing the actual per-sample math that no native Web Audio node can perform. All run on the browser's dedicated audio rendering thread.

4.1 dynamics-worklet.js — Compressor/Limiter core

A feed-forward, zero-look-ahead compressor (used by the Compressor pedal, the Limiter pedal, the always-on output limiter is a WaveShaper instead — see §1.4 — and the Dire Straits amp's built-in compressor). For every sample: computes the peak level across all input channels together (stereo-linked, avoiding image-shift), converts to dB, computes a target gain-reduction using a quadratic soft-knee curve around the threshold, then smooths that gain-reduction value itself (not the level) with separate exponential attack/release coefficients — attack while the reduction is deepening, release while it's recovering toward 0dB. This is the standard causal digital compressor topology (as opposed to a look-ahead design), so it reacts to the signal as it arrives with zero added latency; the trade-off is that a very sharp transient can overshoot slightly before gain catches up.

4.2 noise-gate-worklet.js — Noise Gate + Envelope Follower

Contains two processors. **NoiseGateProcessor**: tracks an amplitude envelope (fast ~3ms attack, ~60ms release), compares it against a threshold, and opens/closes a gain multiplier with independent fast-open (~5ms) / user-configurable-release timing, plus a "hold" counter so the gate doesn't chatter shut during a brief dip in a sustained note. **EnvelopeFollowerProcessor**: the same envelope-tracking technique, but outputs the envelope itself as an audio-rate control signal (0–1) instead of gating — this is what drives the Envelope Filter pedal's "touch wah" response.

4.3 spectral-denoise-worklet.js — Adaptive Spectral Noise Reduction

The most involved piece of DSP in the project. This is classical spectral subtraction / Wiener-style gain masking — not a neural network — but genuinely adaptive: it continuously learns the ambient noise spectrum with no manual "capture profile" step. See §6 for full detail; it's documented separately since it's one of the two Noise Reduction Systems the user asked about.

4.4 bitcrusher-worklet.js — Bit Crusher

True sample-and-hold: a hold counter driven by the "Rate Reduce" parameter causes the processor to only re-sample the input every N samples, holding the last sampled value in between (genuine sample-rate reduction, not a filter approximation of it); the held value is separately quantized to 2^{bits} discrete levels (bit-depth reduction). The two effects are independent knobs (Bits, Rate Reduce) blended against the dry signal via Mix.

4.5 pitch-worklet.js — Octaver & Pitch Shifter

Documented in §2.6 above (Octaver: zero-crossing frequency division; Pitch Shifter: dual-tap delay-line/granular shifting with triangle-window crossfading).

4.6 pluck-worklet.js — Karplus-Strong plucked-string synthesis

Used only for the tuner's per-string reference-tone "play" button, not the guitar signal path. Classic Karplus-Strong physical modeling: a short buffer of lightly-smoothed noise (length = exactly one period of the target frequency, so pitch is sample-accurate) is recirculated through a simple 2-tap averaging + damping filter every time it loops, which progressively removes high-frequency content the same way a real plucked string's harmonics decay fastest — producing a convincing acoustic-guitar-like pluck from pure synthesis, no sampled

audio. Damping is tuned slightly per-string so thinner/higher strings decay faster, matching real string physics. The processor self-terminates once it has decayed to silence.

5. Impulse Response Synthesis (js/ir-synth.js)

Reverb "spaces" and speaker-cabinet responses are both implemented as `ConvolverNodes`, which need an audio buffer (an "impulse response", or IR) to convolve the signal against. Rather than shipping recorded IR sample files, every IR here is **procedurally rendered at runtime** using an `OfflineAudioContext`: a burst of white noise is shaped by a small filter graph (highpass/lowpass/peaking filters, short delay taps for early reflections, an exponential decay envelope), then rendered offline into a real `AudioBuffer` that behaves exactly like a convolution reverb IR would. This keeps the app 100% self-contained/offline with zero fetched assets. Results are cached by their input parameters (sample rate + config) since the "Signal Preview" oscilloscope re-runs every pedal's `createNodes()` on every knob movement, and re-rendering an IR every time would be wasteful.

5.1 Reverb types (used by the Reverb pedal)

- **Room** (0.9s decay) — noise through a 3.5kHz lowpass and a gentle low-shelf boost; the shortest, driest, most contained of the four.
- **Plate** (1.8s) — highpassed at 150Hz with a peaking "shimmer" boost around 3.2kHz, characteristic of a metal-plate reverb's brighter, denser reflection density.
- **Hall** (3.2s, longest) — lowpassed at 6kHz with two explicit early-reflection delay taps (21ms and 37ms) mixed in before the diffuse tail, simulating discrete first reflections off nearby surfaces in a large space.
- **Spring** (1.2s) — a resonant 2.2kHz bandpass plus a short feedback comb-filter delay (6ms), recreating a spring tank's characteristic metallic "boing" resonance and comb-filtered coloration.

5.2 Cabinet impulse responses (used by every amp)

Each amp config specifies cabinet parameters (resonance frequency/Q, lowpass/highpass corner frequencies, decay time, number of reflections). The IR is built from noise through a highpass, a lowpass, and a resonant peaking filter (the speaker's characteristic frequency bump), plus 1–3 short delayed/attenuated taps of that resonant signal simulating early cabinet reflections, all under a fast exponential decay envelope (typically 12–23ms — far shorter than a room/hall reverb, matching how quickly a real speaker cabinet's own resonance dies out).

5.3 Acoustic body impulse response

A 0.35-second IR built from noise through a chain of peaking filters tuned to emulate an acoustic guitar body's resonant characteristics: a low "boom" peak (~100Hz), a lower-mid peak (~220Hz), a "wood resonance" peak (~480Hz), a high-shelf "airiness" boost (~6kHz), and a final lowpass — used by the Acoustic Sim block described in §3.3.

6. Noise Reduction Systems

Two independent, always-in-the-graph noise-handling stages sit right after the input gain, before the user's pedal chain (see the signal-flow diagram in §1.1). Both can be toggled on/off from the UI, and both are implemented as `AudioWorkletProcessors` rather than native nodes, since neither behavior exists as a built-in Web Audio node.

6.1 Spectral Denoiser (`js/spectral-denoise-worklet.js`)

A real-time adaptive spectral noise reduction algorithm — classical **spectral subtraction / Wiener-style gain masking**, not a neural network, but genuinely automatic: it continuously learns the ambient noise spectrum with no manual "capture noise profile" step required from the user.

How it works, step by step:

- Runs a **512-sample Short-Time Fourier Transform (STFT)** with a 256-sample (50%) hop, using a hand-written radix-2 Cooley–Tukey FFT (no external DSP library) with pre-computed bit-reversal and twiddle-factor tables. The 256-sample hop is exactly $2\times$ the browser's 128-sample render quantum, so buffering aligns cleanly across successive process() calls.
- Each 512-sample analysis frame is windowed with a periodic Hann window (denominator N , not $N-1$), which is what makes 50%-overlap-add reconstruction sum back to a constant gain with no separate normalization pass needed.
- After each FFT, the algorithm computes the magnitude spectrum, compares the current frame's total energy to the currently-tracked noise-floor estimate, and updates that noise-floor estimate *adaptively*: for the first 15 frames ("bootstrap"), or whenever the current frame looks quiet (energy ratio below $1.5\times$ the noise floor), the noise spectrum is updated quickly (90/10 exponential blend) per frequency bin; when the frame looks loud (i.e., you're playing), the noise floor is only allowed to creep *down* very slowly (0.997/0.003 blend) and never up — so a sustained loud note is never mistaken for rising noise and "learned" as the new noise floor.
- A per-bin gain mask is computed via oversubtracted spectral subtraction: $\text{gain} = \sqrt{\max(\text{floor}^2, (\text{power} - \text{oversub} \times \text{noisePower}) / \text{power})}$, where both the oversubtraction factor and the noise floor are directly controlled by the user's "Strength" knob (0–100%): higher strength subtracts more aggressively and allows the gain floor to sit lower.
- The raw gain mask is then **smoothed across neighboring frequency bins** (a simple 3-tap [0.25, 0.5, 0.25] blur) to avoid isolated bins creating audible "tinkling"/musical-noise artifacts, and separately **smoothed across time** per bin with asymmetric attack (~8ms, opens fast on note attacks) and release (~30ms, closes slowly to avoid audible gain "chatter").
- The masked spectrum is reconstructed into a full complex spectrum (using conjugate symmetry, since the input is real-valued) and transformed back to the time domain with an inverse FFT, then reassembled via standard overlap-add across the 50%-overlapping frames.
- **Performance note:** because this node sits permanently in the signal path, when Denoise is switched off (the default) the processor skips all of the above entirely and just copies input to output — so users who never enable it pay zero CPU cost for it, at the price of the noise-floor estimate needing a short (~15-frame, ~80ms) re-bootstrap the next time it's turned on.

6.2 Noise Gate (`js/noise-gate-worklet.js`)

A traditional envelope-follower noise gate, running after the spectral denoiser in the chain. It tracks the input's amplitude envelope with a fast attack (~3ms) and moderate release (~60ms) time constant, compares that envelope against a user-adjustable Threshold (in dB, converted once to a linear comparison value to avoid a per-sample $\log_{10}$ call), and drives a smoothly-interpolated gain multiplier that opens quickly (~5ms, fixed)

whenever the envelope crosses the threshold and closes at a user-adjustable Release rate once it falls back below it. A separate "Hold" time (in ms, also user-adjustable) keeps the gate held open for a short window after the signal dips below threshold, preventing the gate from chattering open/closed during natural micro-dips within a sustained note or chord (e.g. between rapid picking attacks).

Together, the two stages address different noise problems: the spectral denoiser continuously removes a learned *hiss/hum floor* even while you're playing, while the gate fully silences the signal during pauses between notes (eliminating pickup hum, amp hiss, or room noise when you're not actively playing) — the classic two-layer approach used in professional guitar noise-reduction hardware (e.g. an ISP Decimator or a Boss NS-2 combined with a subtractive EQ notch).

7. Recording System (js/looper.js, js/wav-encoder.js)

The `Looper` class implements record/playback/loop functionality for both the processed guitar signal and, optionally, a second synchronized audio source (e.g. a backing track playing in another browser tab).

7.1 Capture

- The guitar signal is tapped post-limiter (the very end of the processing chain, see §1.1) via a `MediaStreamAudioDestinationNode` (`mediaStreamDest`, created once in the static graph). Its `.stream` is fed straight into a native `MediaRecorder`, which encodes it live to WebM/Opus (or WebM/plain, or Ogg/Opus, whichever the browser supports first, checked via `MediaRecorder.isTypeSupported()`).
- Optionally, a *second*, fully independent `MediaRecorder` can simultaneously capture system/tab audio via `navigator.mediaDevices.getDisplayMedia({video:true, audio:true})` — the browser's screen/tab-share picker (audio-only capture alone isn't offered by this API, so a video track is requested but never read/rendered anywhere, just kept alive so Chrome doesn't kill the audio track when video is stopped). If the user cancels the picker, denies permission, or their browser/OS doesn't provide a shareable audio track, this silently falls back to a guitar-only recording rather than failing outright.
- Both recorders are started back-to-back so the two takes stay closely time-aligned.

7.2 Decode & playback

- On stop, each recorder's chunk list is combined into a `Blob` and decoded back into a full-quality `AudioBuffer` via `ctx.decodeAudioData()` — converting from the compressed WebM/Opus container back to raw PCM samples in memory.
- Playback uses `AudioBufferSourceNode`, which (unlike an HTML `<audio>` element) has no native seek or `currentTime` property, so the `Looper` tracks playback position manually: it records the `AudioContext` clock time playback last (re)started at plus the buffer offset it started from, and computes current position from the two. Seeking, pausing, and looping all work by stopping and restarting a fresh source node at the right offset, since a source node can only be played once.
- When both a guitar take and a screen-audio take exist, the user can toggle between hearing just the guitar or a mix of both ("source": 'guitar' | 'both'), each played through its own synchronized source node into a shared monitor gain.

7.3 WAV export (js/wav-encoder.js)

`MediaRecorder` cannot output WAV directly (only compressed containers like WebM/Ogg), so exporting a clean, universally-compatible WAV file is done by hand: `audioBufferToWavBlob()` writes a 44-byte RIFF/WAVE header directly into an `ArrayBuffer` (format tag, channel count, sample rate, byte rate, block align, bit depth, all little-endian per the RIFF spec), then walks the decoded `AudioBuffer`'s float samples (range $-1..1$) per channel, clamps them, and writes each as a 16-bit signed PCM integer. This is a from-scratch binary file writer with no encoding library involved.

8. Supporting Subsystems

8.1 Chromatic Tuner (js/tuner.js)

Pitch detection uses **autocorrelation with parabolic interpolation** — a standard, well-established real-time instrument-tuner technique. The input buffer is trimmed to its loudest central region (silence at the edges is dropped), then for each candidate lag up to roughly the period of the lowest note the tuner needs to detect (~55Hz), the buffer is correlated against a copy of itself shifted by that lag; the lag with the strongest self-similarity corresponds to the fundamental period. Parabolic interpolation around the peak lag then refines the estimate to sub-sample precision, run at roughly 30fps. Detected frequency is converted to note name + cents-off either against the nearest chromatic note, or (in "focused" per-string mode) against one specific target string's frequency, so a badly out-of-tune string still reads as flat/sharp of the intended string rather than jumping to a different note name entirely. The reference "play tone" button per string uses the Karplus-Strong pluck synth described in §4.6.

8.2 Drum Machine (js/drum-kit.js, js/beat-presets.js)

All drum sounds are synthesized, not sampled — consistent with the project's fully-offline, no-third-party-asset design. Each pad is described declaratively as either a pitched **'tone'** (a sine oscillator with an exponential pitch-drop envelope, used for kick/tom/rim) or a filtered **'noise'** burst (band/high-pass filtered white noise, used for snare/hats/clap/crash), and a **'layered'** type composites several of either kind, optionally offset in time (e.g. a clap is three staggered bandpassed noise bursts). Multiple named drum kits (e.g. "Acoustic Kit") simply supply different parameter sets for the same synthesis engine. A separate beat-presets/pattern-player module sequences these pads against a tempo clock for backing-track style practice.

8.3 UI Rendering (js/ui.js)

Pedal and amp "stompbox" cards are rendered from a single shared HTML `<template>`, cloned once per chain entry, with knob rows built dynamically from each effect's `params[]` definition (a rotary dial UI element synchronized to a hidden native `<input type="range">` for accessibility/keyboard support, plus click-to-type-exact-value editing). Pedal reordering is a fully custom Pointer-Events-based drag implementation (not native HTML5 drag-and-drop, which the codebase found too inconsistent across browsers to script/test reliably) using a dashed placeholder element to preview the drop position. The patch cables drawn between pedals are hand-built SVG paths with a layered fake-material rendering (ground shadow, fuzzy fiber halo, dark rim, a woven-pattern fill, under-shade, and specular highlight) to visually read as braided cable rather than a flat line, referenced from real cable photos but built entirely from CSS/SVG gradients and patterns — no bitmap textures are used anywhere in the UI; every visual element (knobs, screws, jacks, footswitches, cables, and the amp/pedal enclosures themselves) is drawn purely with CSS gradients, box-shadows, and generated SVG.